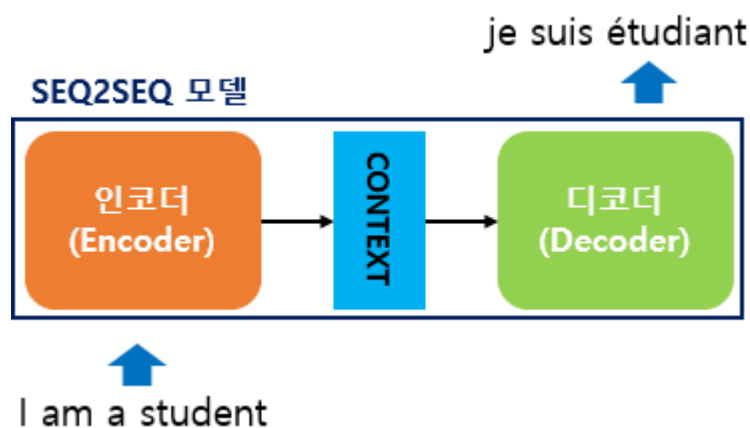
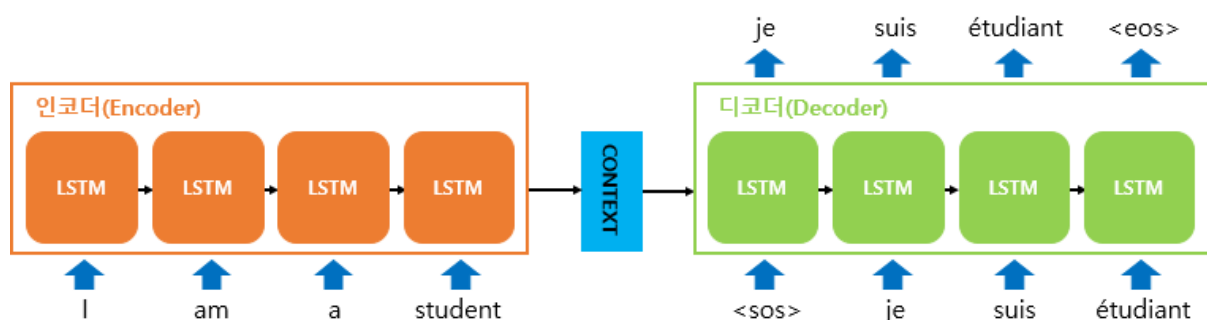


seq2seq

RNN과 LSTM처럼 Sequence Data에 대해서도 상당히 좋은 성능을 보이는 모델들은 있었다. 그러나 이들의 문제점은 Input으로 Fixed - Length Vector를 넘겨주어야 한다. 가변적인 길이를 가질 수 밖에 없는 언어의 특성상 이 부분을 해소하는 것이 어려웠는데 해당 논문은 이 부분을 해소하고자 했다.



seq2seq는 크게 인코더와 디코더라는 두 개의 모듈로 구성됩니다. 인코더는 입력 문장의 모든 단어들을 순차적으로 입력받은 뒤에 마지막에 이 모든 단어 정보들을 압축해서 하나의 벡터로 만드는데, 이를 컨텍스트 벡터(context vector)라고 합니다. 입력 문장의 정보가 하나의 컨텍스트 벡터로 모두 압축되면 인코더는 컨텍스트 벡터를 디코더로 전송합니다. 디코더는 컨텍스트 벡터를 받아서 번역된 단어를 한 개씩 순차적으로 출력합니다.

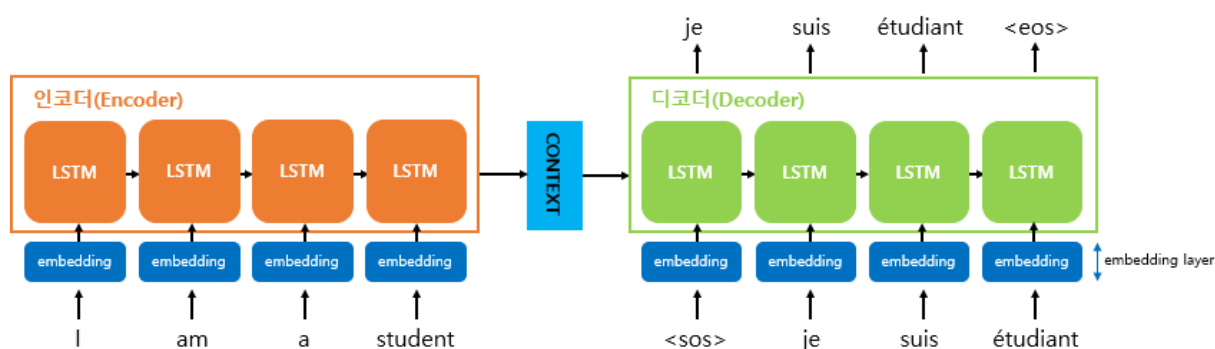


인코더 아키텍처와 디코더 아키텍처의 내부는 사실 두 개의 RNN 아키텍처입니다. 입력 문장을 받는 RNN 셀을 인코더라고 하고, 출력 문장을 출력하는 RNN 셀을 디코더라고 합니다. 우선 인코더를 자세히보면, 입력 문장은 단어 토큰화를 통해서 단어 단위로 쪼개지고 단어 토큰 각각은 RNN 셀의 각 시점의 입력이 됩니다. 인코더 RNN 셀은 모든 단어를 입력받은 뒤에 **인코더 RNN 셀의 마지막 시점의 은닉 상태**를 디코더 RNN 셀로 넘겨주는데 이를 컨

텍스트 벡터라고 합니다. 컨텍스트 벡터는 **디코더 RNN 셀의 첫번째 은닉 상태**에 사용됩니다.

디코더는 기본적으로 RNNLM(RNN Language Model)입니다. RNNLM의 개념을 기억하고 있다면 좀 더 이해하기 쉽습니다. 디코더는 초기 입력으로 문장의 시작을 의미하는 심볼 <sos>가 들어갑니다. 디코더는 <sos>가 입력되면, 다음에 등장할 확률이 높은 단어를 예측합니다. 첫 번째 시점(time step)의 디코더 RNN 셀은 다음에 등장할 단어로 je를 예측하였습니다. 첫 번째 시점의 디코더 RNN 셀은 예측된 단어 je를 다음 시점의 RNN 셀의 입력으로 입력합니다. 그리고 두 번째 시점의 디코더 RNN 셀은 입력된 단어 je로부터 다시 다음에 올 단어인 suis를 예측하고, 또 다시 이것을 다음 시점의 RNN 셀의 입력으로 보냅니다. 디코더는 이런 식으로 기본적으로 다음에 올 단어를 예측하고, 그 예측한 단어를 다음 시점의 RNN 셀의 입력으로 넣는 행위를 반복합니다. 이 행위는 문장의 끝을 의미하는 심볼인 <eos>가 다음 단어로 예측될 때까지 반복됩니다. **지금 설명하는 것은 테스트 과정** 동안의 이야기입니다.

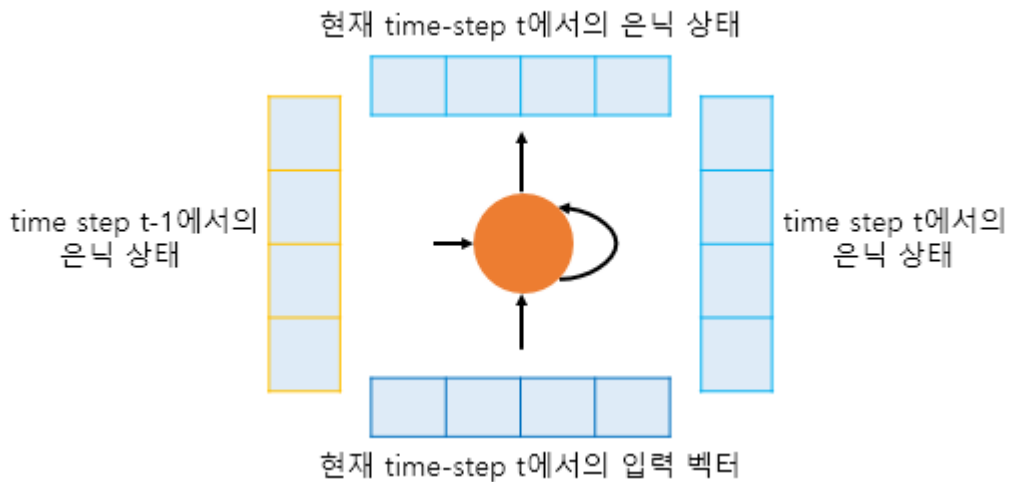
seq2seq는 훈련 과정과 테스트 과정(또는 실제 번역기를 사람이 쓸 때)의 작동 방식이 조금 다릅니다. 훈련 과정에서는 디코더에게 인코더가 보낸 컨텍스트 벡터와 실제 정답인 상황인 <sos> je suis étudiant를 입력 받았을 때, je suis étudiant <eos>가 나와야 된다고 정답을 알려주면서 훈련합니다. 이에 대해서는 뒤에 **교사 강요(teacher forcing)**를 설명하면서 재언급하겠습니다. 반면 테스트 과정에서는 앞서 설명한 과정과 같이 디코더는 오직 컨텍스트 벡터와 <sos>만을 입력으로 받은 후에 다음에 올 단어를 예측하고, 그 단어를 다음 시점의 RNN 셀의 입력으로 넣는 행위를 반복합니다. 즉, 앞서 설명한 과정과 위의 그림은 테스트 과정에 해당됩니다. 이번에는 입, 출력에 쓰이는 단어 토큰들이 있는 부분을 좀 더 확대해보겠습니다.



기계는 텍스트보다 숫자를 잘 처리합니다. 자연어 처리에서 텍스트를 벡터로 바꾸는 방법으로 주로 워드 임베딩이 사용된다. seq2seq에서 사용되는 모든 단어들은 임베딩 벡터로 변환 후 입력으로 사용됩니다.

-		0.157
		-0.25
		0.478
		-0.78
am		0.78
		0.29
		-0.96
		0.52
a		0.75
		-0.81
		0.96
		0.12
student		0.88
		-0.17
		0.29
		0.48

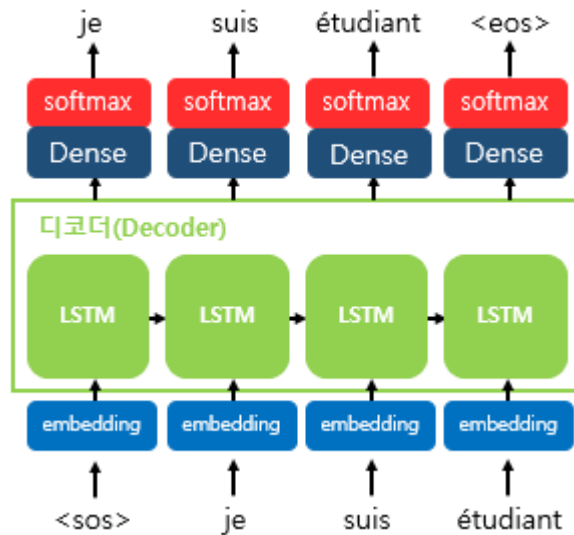
하나의 RNN 셀은 각각의 시점(time step)마다 두 개의 입력을 받습니다.



현재 시점(time step)을 t 라고 할 때, RNN 셀은 $t-1$ 에서의 은닉 상태와 t 에서의 입력 벡터를 입력으로 받고, t 에서의 은닉 상태를 만듭니다. 이때 t 에서의 은닉 상태는 바로 위에 또 다른 은닉층이나 출력층이 존재할 경우에는 위의 층으로 보내거나, 필요없으면 값을 무시할 수 있습니다. 그리고 RNN 셀은 다음 시점에 해당하는 $t+1$ 의 RNN 셀의 입력으로 현재 t 에서의 은닉 상태를 입력으로 보냅니다.

현재 시점 t 에서의 은닉 상태는 과거 시점의 동일한 RNN 셀에서의 모든 은닉 상태의 값들의 영향을 누적해서 받아온 값이라고 할 수 있습니다. 그렇기 때문에 컨텍스트 벡터는 사실 인코더에서의 마지막 RNN 셀의 은닉 상태값을 말하는 것이며, 이는 입력 문장의 모든 단어 토큰들의 정보를 요약해서 담고있다고 할 수 있습니다.

디코더는 인코더의 마지막 RNN 셀의 은닉 상태인 컨텍스트 벡터를 첫번째 은닉 상태의 값으로 사용합니다. 디코더의 첫번째 RNN 셀은 이 첫번째 은닉 상태의 값과, 현재 t 에서의 입력값인 $\langle \text{sos} \rangle$ 로부터, 다음에 등장할 단어를 예측합니다. 그리고 이 예측된 단어는 다음 시점인 $t+1$ RNN에서의 입력값이 되고, 이 $t+1$ 에서의 RNN 또한 이 입력값과 t 에서의 은닉 상태로부터 $t+1$ 에서의 출력 벡터. 즉, 또 다시 다음에 등장할 단어를 예측하게 될 것입니다.



seq2seq 모델은 선택될 수 있는 모든 단어로부터 하나의 단어를 골라서 예측해야 하므로 디코더에서 각 시점(time step)의 RNN 셀에서 출력 벡터가 나오면, 해당 벡터는 소프트맥스 함수를 통해 출력 시퀀스의 각 단어별 확률값을 반환하고, 디코더는 출력 단어를 결정합니다.

이때 구체적으로는 Decoder를 통해 Target 문장을 만들 때에 Beam Search 기법이 사용된다.

이에 대한 설명은 다음과 같다.

가장 가능성이 높은 출력 시퀀스를 디코딩하는 것은 그 가능성에 기초하여 가능한 모든 출력 시퀀스를 검색하는 것을 포함한다. 이때 vocab의 크기는 수백 수천, 수백만이 될때가 있다. 따라서

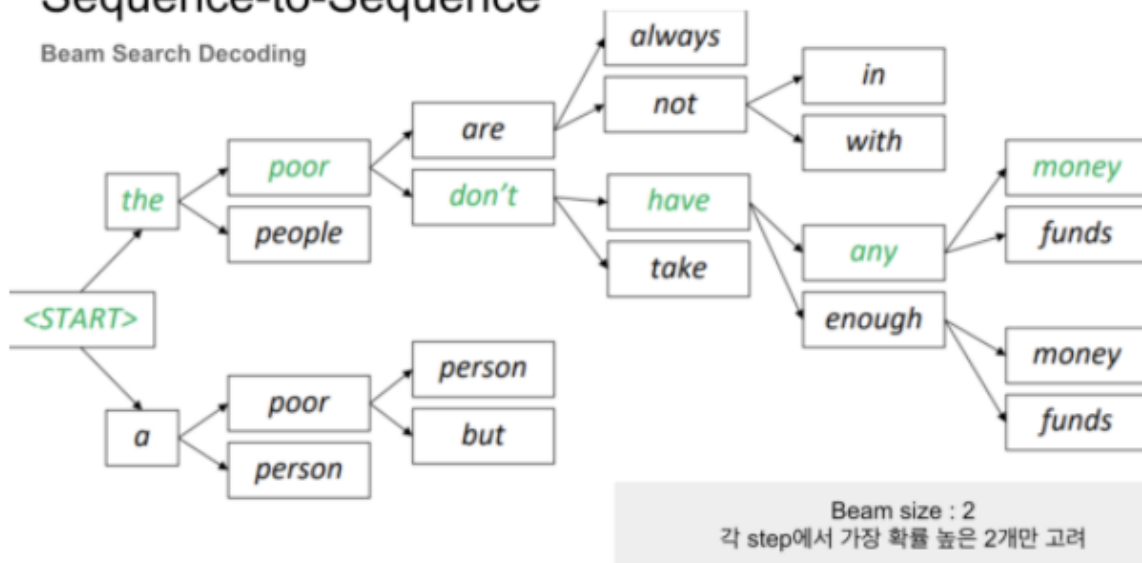
탐색문제는 출력 시퀀스에 지수적이기 때문에 완전히 탐색하기 어렵다. 그래서 **휴리스틱 탐색** 방법이 사용한다. 휴리스틱은 보다 근사적이거나 충분히 디코딩된 출력 시퀀스를 반환한다.

단어들의 후보 시퀀스들은 그들의 우도(likelihood)에 따라 점수화 되고, 다음 텍스트를 예측하는 것에 **Greedy Search**와 **Beam Search**를 일반적으로 사용한다.

Sequence-to-Sequence

출처 : cs-224n

Beam Search Decoding



빔 탐색은 모든 가능한 다음 스텝들로 확장하고, k 가 사용자 지정 파라미터 이고, 빔의 숫자 또는 확률 시퀀스에서 병렬 탐색들을 조절가능한 곳에서 가능한 k 를 유지하려고 한다.

그리디 탐색의 경우 빔이 1인 탐색과 같다. 빔의 수는 일반적으로 5 또는 10을 사용하고, 빔이 클수록 타겟 시퀀스가 맞을 확률이 높지만 디코딩 속도가 떨어지게 된다.

NMT에서 빔 탐색 디코더를 통해 문장을 번역하는것은 학습된 NMT 모델의 조건부 확률의 최대화하는 번역을 찾는것이다. 빔 탐색전략은 고정된 숫자를 유지하면서 왼쪽->오른쪽으로 이동하면서 단어를 생성한다. 빔 크기를 증가시면 번역 성능은 높아지나 디코딩 속도는 떨어진 다.

Beam Search Strategies for Neural Machine Translation, 2017

탐색 절차는 각 후보 단어들에 대해서 최대 길이에 도달할때 EOS 토큰에 도달하거나 임계값 우도에 도달할때 탐색을 개별적으로 멈출 수 있다.

이 논문에서 가장 특이했던 점은 Encoder를 통해 문장을 읽을 때 역순으로 읽는 점이다. 이를 통하면 순방향으로 읽을 때보다 성능이 훨씬 좋게 나온다는데 왜인지는 구체적으로 궁금하다...

이에 대해 처음 단어들을 마지막에 읽게 되니 초반부에 대한 Gradient가 적게 소실되게 되고 이를 통해 첫부분의 Output을 내놓을 때 정확도를 높일 수 있으므로 이후에 차용되는 Beam Search 방식을 통해 SGD를 통한 목적함수 최대화를 효과적으로 이루지 않았나라고 가설을 내놓았다.